

Financial Transaction Fraud Detection Using Machine Learning

Student Details

Name: Harsh Pramodh

Course: B.Com (Business Analytics)

College: Symbiosis College of Arts and Commerce, Pune

Abstract

Financial fraud detection is a critical challenge for banks and financial institutions. This project aims to identify fraudulent transactions using Machine Learning techniques. A dataset containing over 54,000 financial transactions was analyzed using Python. Exploratory Data Analysis (EDA) was performed to identify fraud patterns, followed by model development using Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN). Model performance was evaluated using Precision, Recall, and F1-Score due to the highly imbalanced nature of the dataset.

Objectives

- Analyze transaction data to identify fraud patterns.
- Perform exploratory data analysis on financial transaction records.
- Build machine learning models for fraud detection.
- Compare model performance using appropriate classification metrics.
- Identify the most effective model for fraud prediction.

Tools & Technologies

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-Learn
- Google Colab

Dataset Overview

Total Transactions: 54,448

Legitimate Transactions: 54,233

Fraudulent Transactions: 215

Fraud Rate: 0.39%

Screenshot 1

df.head()

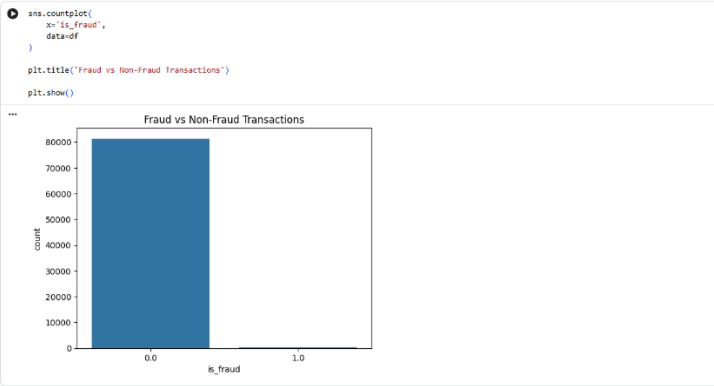
	Unnamed: 0	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street	...	lat	long	city_pop	job	dob
0	0	2020-06-21 12:14:25	2291163933867244	fraud_Kirlin and Sons	personal_care	2.86	Jeff	Elliott	M	351 Darlene Green	...	33.9659	-80.9355	333497.0	Mechanical engineer	1968-03-19
1	1	2020-06-21 12:14:33	3573030041201292	fraud_Sporer-Keebler	personal_care	29.84	Joanne	Williams	F	3638 Marsh Union	...	40.3207	-110.4360	302.0	Sales professional, IT	1990-01-17
2	2	2020-06-21 12:14:53	3598215285024754	fraud_Swaniawski, Nitzsche and Welch	health_fitness	41.28	Ashley	Lopez	F	9333 Valentine Point	...	40.6729	-73.5365	34496.0	Librarian, public	1970-10-21
3	3	2020-06-21 12:15:15	3591919803438423	fraud_Haley Group	misc_pos	60.05	Brian	Williams	M	32941 Krystal Mill Apt. 552	...	28.5697	-80.8191	54767.0	Set designer	1987-07-25
4	4	2020-06-21 12:15:17	3526826139003047	fraud_Johnston-Casper	travel	3.19	Nathan	Massey	M	5783 Evan Roads Apt. 465	...	44.2529	-85.0170	1126.0	Furniture designer	1955-07-06

5 rows x 23 columns

Exploratory Data Analysis (EDA)

Fraud Distribution Analysis

Screenshot 2



Interpretation

The dataset is highly imbalanced, with fraudulent transactions accounting for only 0.39% of all transactions. This mirrors real-world financial systems where fraudulent activities occur much less frequently than legitimate transactions.

Transaction Amount Distribution

Screenshot 3

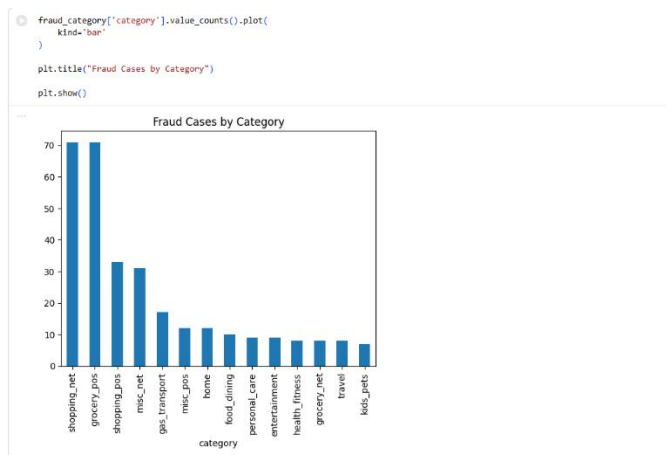
Insert Transaction Amount Histogram

Interpretation

Most transactions occur at lower monetary values, while high-value transactions are less common. The distribution is positively skewed, indicating concentration of transactions within lower ranges.

Fraud by Transaction Category

Screenshot 4

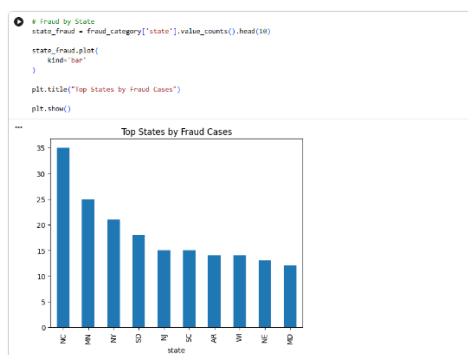


Interpretation

Fraud occurrences are concentrated within specific merchant categories, indicating that certain transaction types are more susceptible to fraudulent activities.

Fraud by State

Screenshot 5



Interpretation

Fraudulent transactions are unevenly distributed across states, suggesting possible geographic differences in fraud occurrence.

Data Preprocessing

The following preprocessing techniques were applied:

- Removed unnecessary identifier columns.
- Handled missing values.
- Encoded categorical variables using Label Encoding.
- Performed train-test split using stratification.
- Standardized features using StandardScaler.

Machine Learning Models

Logistic Regression

Screenshot 6

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(
    max_iter=1000,
    class_weight='balanced'
)

lr.fit(X_train, y_train)

LogisticRegression
LogisticRegression(class_weight='balanced', max_iter=1000)

lr_pred = lr.predict(X_test)

print(classification_report(y_test, lr_pred))
```

	precision	recall	f1-score	support
0.0	1.00	0.91	0.95	16271
1.0	0.03	0.80	0.06	61
accuracy			0.91	16332
macro avg	0.52	0.86	0.51	16332
weighted avg	1.00	0.91	0.95	16332

Results

Precision: 0.03

Recall: 0.80

F1-Score: 0.06

Interpretation

The initial Logistic Regression model struggled due to severe class imbalance. Applying class weights improved recall significantly, allowing the model to detect most fraudulent transactions. However, precision remained low due to a large number of false positives.

Decision Tree

Screenshot 7

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    random_state=42
)

dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

print(classification_report(y_test, dt_pred))
```

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	16271
1.0	0.61	0.64	0.62	61
accuracy			1.00	16332
macro avg	0.80	0.82	0.81	16332
weighted avg	1.00	1.00	1.00	16332

Results

Precision: 0.61

Recall: 0.64

F1-Score: 0.62

Interpretation

Decision Tree achieved the most balanced performance among all models. It effectively identified fraudulent transactions while maintaining a relatively low false positive rate.

K-Nearest Neighbors (KNN)

Screenshot 8

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=5
)

knn.fit(X_train, y_train)

knn_pred = knn.predict(X_test)

print(classification_report(y_test, knn_pred))
```

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	16271
1.0	0.69	0.41	0.52	61
accuracy			1.00	16332
macro avg	0.85	0.70	0.76	16332
weighted avg	1.00	1.00	1.00	16332

Results

Precision: 0.69

Recall: 0.41

F1-Score: 0.52

Interpretation

KNN produced the highest precision but missed a larger number of fraudulent transactions, resulting in lower recall.

Model Comparison

Screenshot 9

```

8 results = pd.DataFrame({
    "model": [
        "Logistic Regression",
        "Decision Tree",
        "KNN"
    ],
    "Precision": [
        0.89,
        0.61,
        0.69
    ],
    "Recall": [
        0.88,
        0.64,
        0.41
    ],
    "F1 Score": [
        0.89,
        0.62,
        0.52
    ]
})
9
results

```

	Model	Precision	Recall	F1 Score
0	Logistic Regression	0.89	0.88	0.86
1	Decision Tree	0.61	0.64	0.62
2	KNN	0.69	0.41	0.52

Next steps: [Generate code with results](#) [New interactive sheet](#)

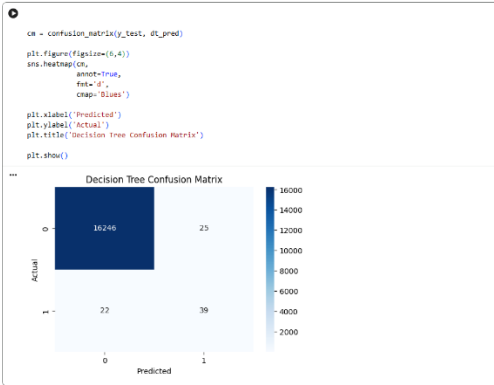
Model	Precision	Recall	F1 Score
Logistic Regression	0.03	0.80	0.06
Decision Tree	0.61	0.64	0.62
KNN	0.69	0.41	0.52

Interpretation

Decision Tree achieved the highest overall F1-Score and provided the best balance between fraud detection and false alarm reduction.

Confusion Matrix Analysis

Screenshot 10



Results:

- True Negatives: 16,246
- False Positives: 25
- False Negatives: 22
- True Positives: 39

Interpretation

The Decision Tree model correctly identified 39 fraudulent transactions while incorrectly flagging only 25 legitimate transactions. This demonstrates a practical balance between fraud detection effectiveness and operational efficiency.

Conclusion

This project successfully applied Machine Learning techniques to detect fraudulent financial transactions. Logistic Regression, Decision Tree, and KNN models were developed and evaluated. Due to severe class imbalance, evaluation focused on Precision, Recall, and F1-Score rather than accuracy alone.

Among the tested models, the Decision Tree classifier delivered the strongest overall performance, achieving an F1-Score of 0.62 while maintaining a balanced tradeoff between fraud detection and false-positive reduction.

The project demonstrates the practical application of Python and Machine Learning in solving real-world financial analytics problems.